```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras import layers, models
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

print("TensorFlow version:", tf.__version__)
```

Insert code cell options

```
TensorFlow version: 2.20.0
```

```
# Load MNIST handwritten digit dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

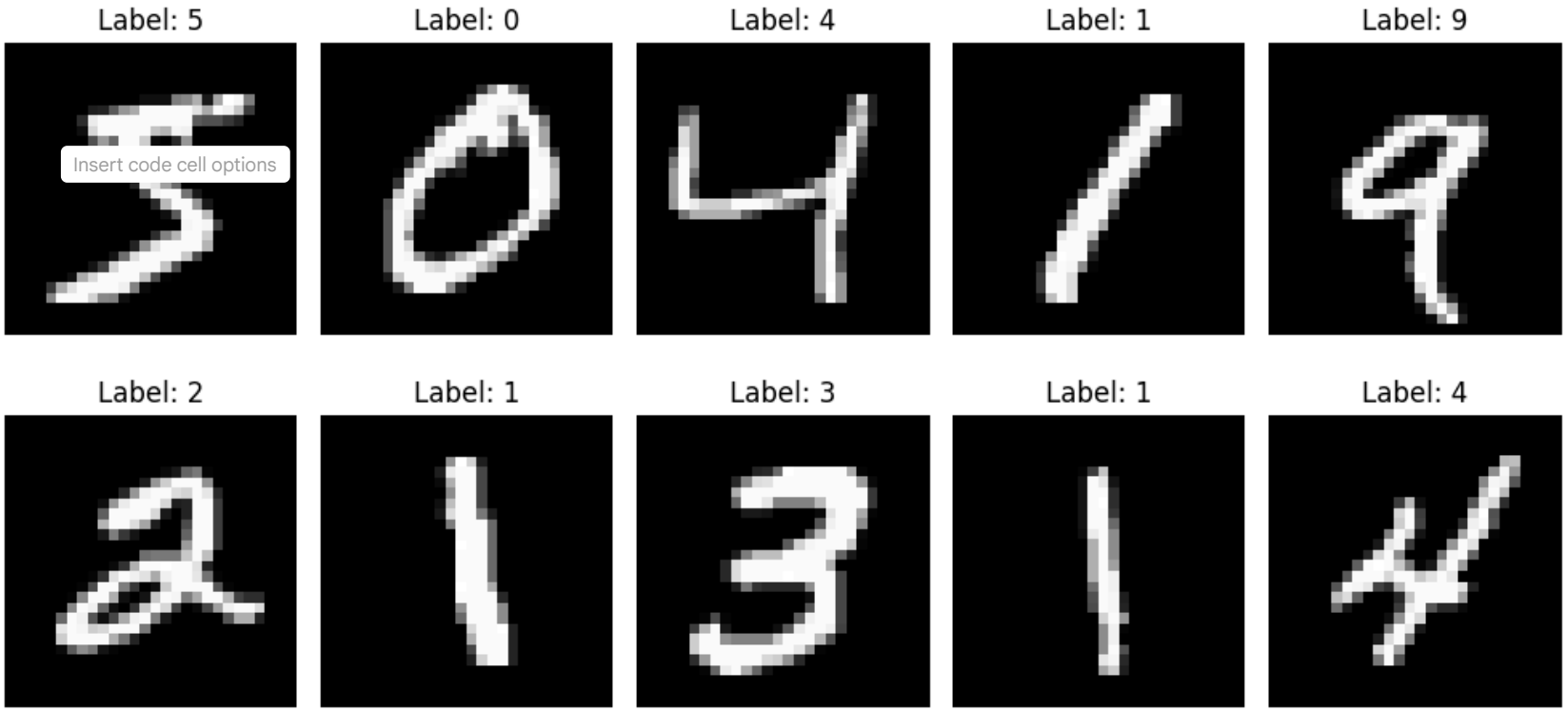
print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)
```

```
plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
# Normalize pixel values from 0-255 to 0-1
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Add channel dimension
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

print("Training shape:", x_train.shape)
print("Testing shape:", x_test.shape)
```

```
Training shape: (60000, 28, 28, 1)
Testing shape: (10000, 28, 28, 1)
```

```
model = models.Sequential([

    # Convolutional layer
    layers.Conv2D(32, (3, 3), activation="relu", input_shape=(28, 28, 1)),
```

```
    # Pooling layer
    layers.MaxPooling2D((2, 2)),

    # Second convolutional layer
    layers.Conv2D(64, (3, 3), activation="relu"),
```

Insert code cell options

```
    # Second pooling layer
    layers.MaxPooling2D((2, 2)),

    # Convert feature maps into a vector
    layers.Flatten(),

    # Fully connected layer
    layers.Dense(128, activation="relu"),

    # Output layer: 10 digits
    layers.Dense(10, activation="softmax")
])

model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` ar
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv Insert code cell options	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully!")
```

```
Model compiled successfully!
```

```
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1,
    verbose=1
)
```

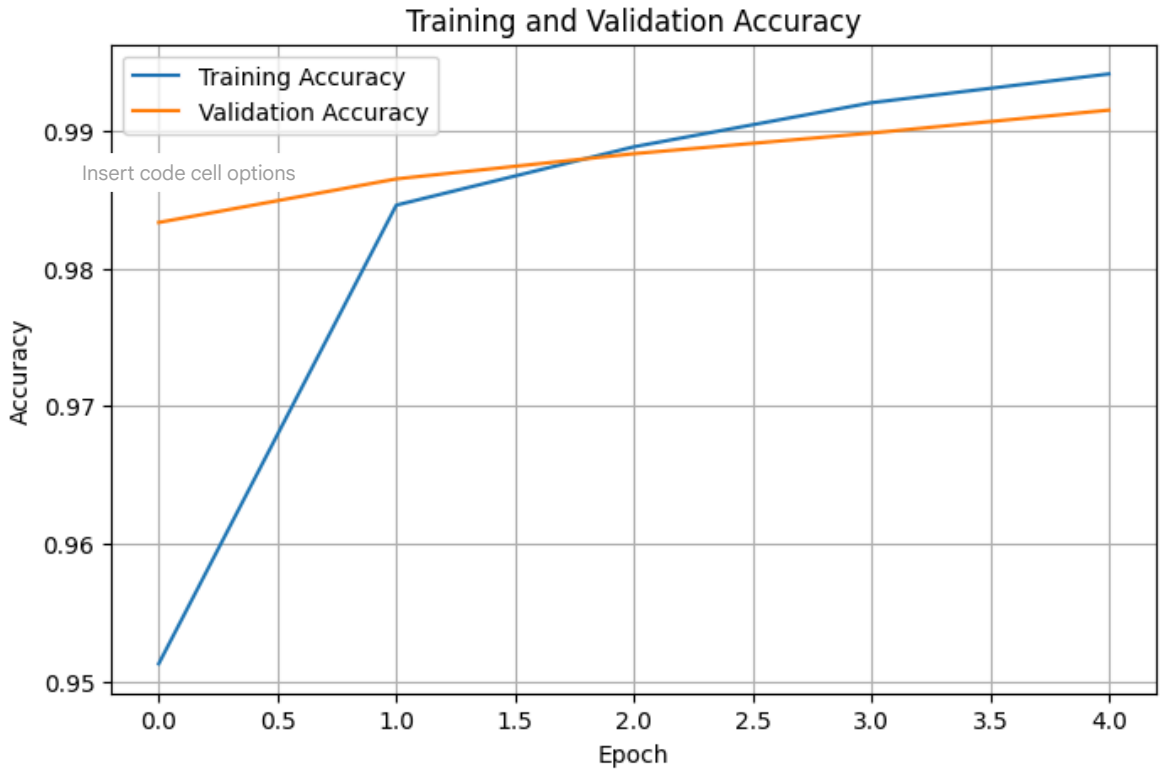
```
Epoch 1/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 45s 52ms/step - accuracy: 0.9513 - loss: 0.1643 - val_accuracy: 0.9833 - val_loss: 0.0543
Epoch 2/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 45s 53ms/step - accuracy: 0.9846 - loss: 0.0497 - val_accuracy: 0.9865 - val_loss: 0.0452
Epoch 3/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 43s 51ms/step - accuracy: 0.9888 - loss: 0.0354 - val_accuracy: 0.9883 - val_loss: 0.0388
Epoch
844/844 ━━━━━━━━━━━━━━━━━━━━ 43s 51ms/step - accuracy: 0.9920 - loss: 0.0251 - val_accuracy: 0.9898 - val_loss: 0.0354
Epoch 5/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 52ms/step - accuracy: 0.9941 - loss: 0.0187 - val_accuracy: 0.9915 - val_loss: 0.0311
```

Insert code cell options

```
plt.figure(figsize=(8, 5))

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

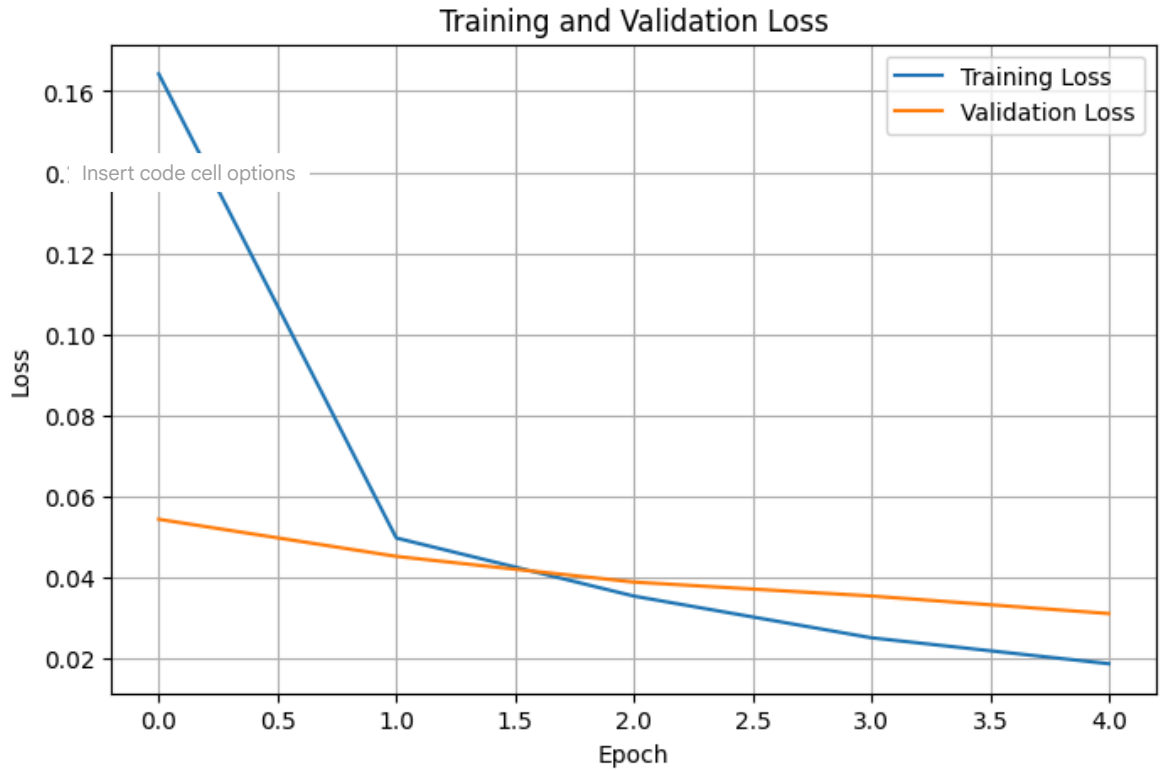
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.grid()
plt.show()
```




```
plt.figure(figsize=(8, 5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid()
plt.show()
```




```
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

```
Test Loss: 0.0302
Test Accuracy: 0.9905
Test Accuracy: 99.05%
```

```
predictions = model.predict(x_test)

# Convert probabilities to predicted class
```

```
y_pred = np.argmax(predictions, axis=1)

print("Predictions generated successfully!")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step
Predictions generated successfully!
```

Insert code cell options

```
print("Classification Report:")
print("=" * 60)

print(classification_report(
    y_test,
    y_pred,
    digits=4
))
```

```
Classification Report:
============================================================
              precision    recall  f1-score   support

           0     0.9919    0.9980    0.9949       980
           1     0.9974    0.9965    0.9969      1135
           2     0.9941    0.9874    0.9908      1032
           3     0.9950    0.9782    0.9865      1010
           4     0.9899    0.9949    0.9924       982
           5     0.9844    0.9888    0.9866       892
           6     0.9979    0.9864    0.9921       958
           7     0.9808    0.9942    0.9874      1028
           8     0.9877    0.9877    0.9877       974
           9     0.9852    0.9921    0.9886      1009

    accuracy                         0.9905     10000
   macro avg     0.9904    0.9904    0.9904     10000
weighted avg     0.9905    0.9905    0.9905     10000
```

```
cm = confusion_matrix(y_test, y_pred)

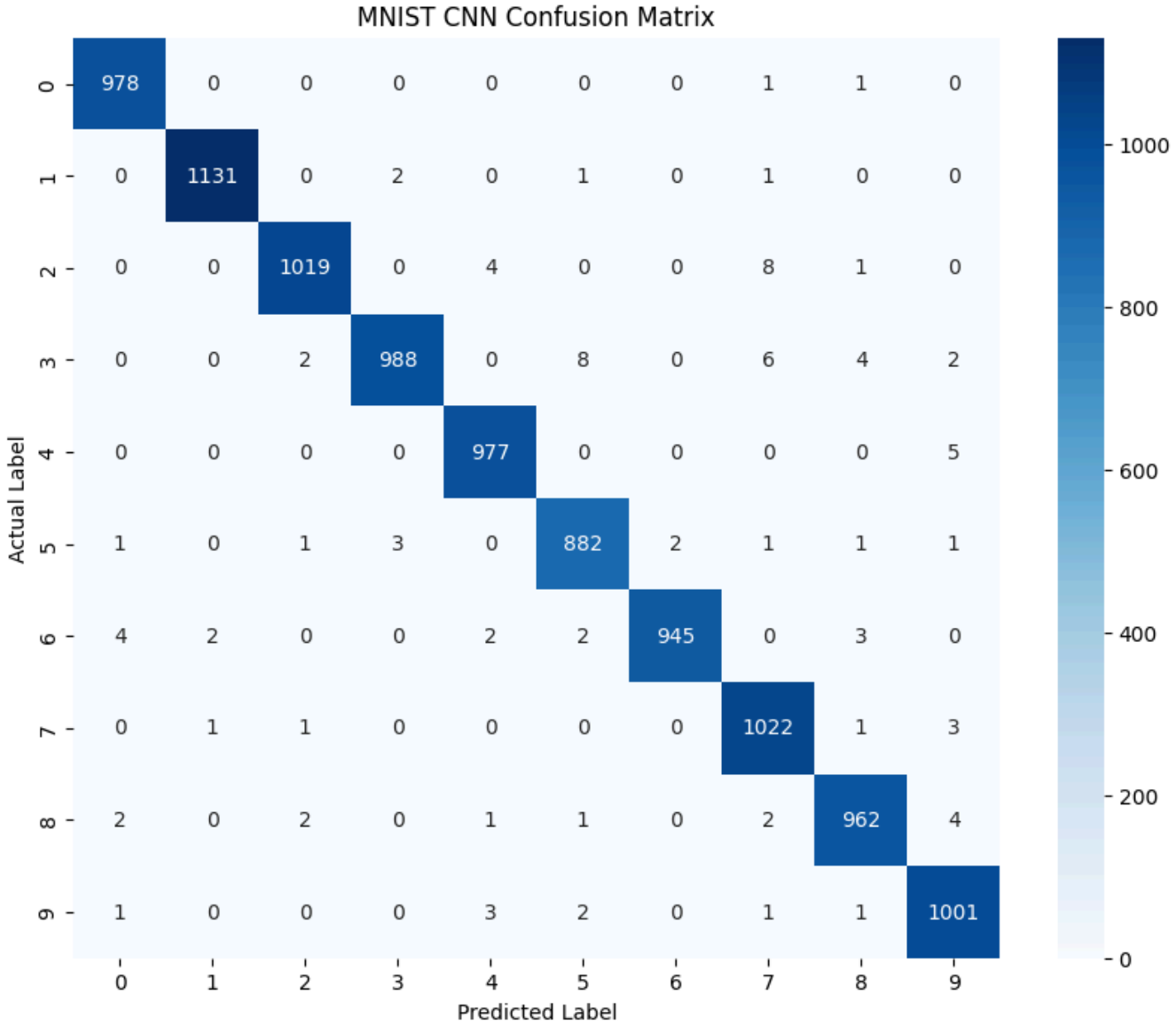
plt.figure(figsize=(10, 8))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
```

```
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("MNIST CNN Confusion Matrix")
plt.sh
```

Insert code cell options



```
index = 25

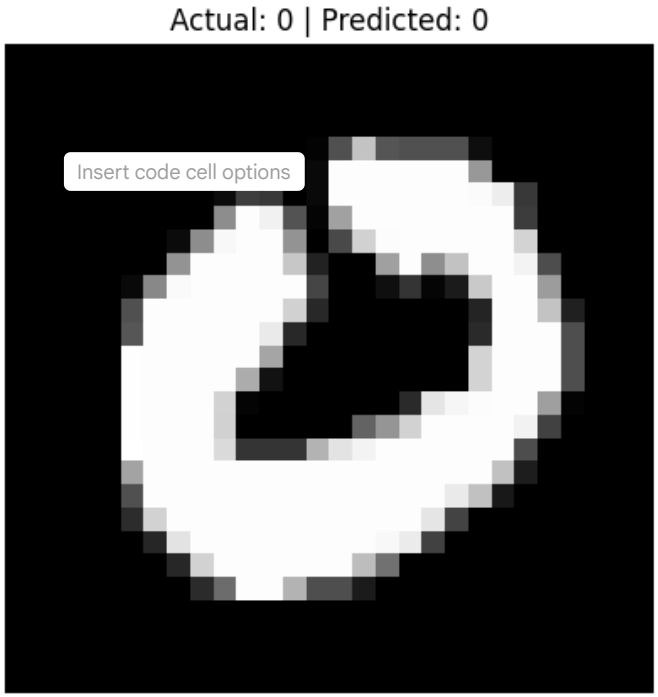
image = x_test[index]
actual_label = y_test[index]

predic Insert code cell options ct(
    image.reshape(1, 28, 28, 1),
    verbose=0
)

predicted_label = np.argmax(prediction)

plt.figure(figsize=(5, 5))
plt.imshow(image.squeeze(), cmap="gray")
plt.title(
    f"Actual: {actual_label} | Predicted: {predicted_label}"
)
plt.axis("off")
plt.show()

print("Actual Label:", actual_label)
print("Predicted Label:", predicted_label)
```




```
Actual Label: 0
Predicted Label: 0
```

```
incorrect = np.where(y_pred != y_test)[0]

print("Number of incorrect predictions:", len(incorrect))

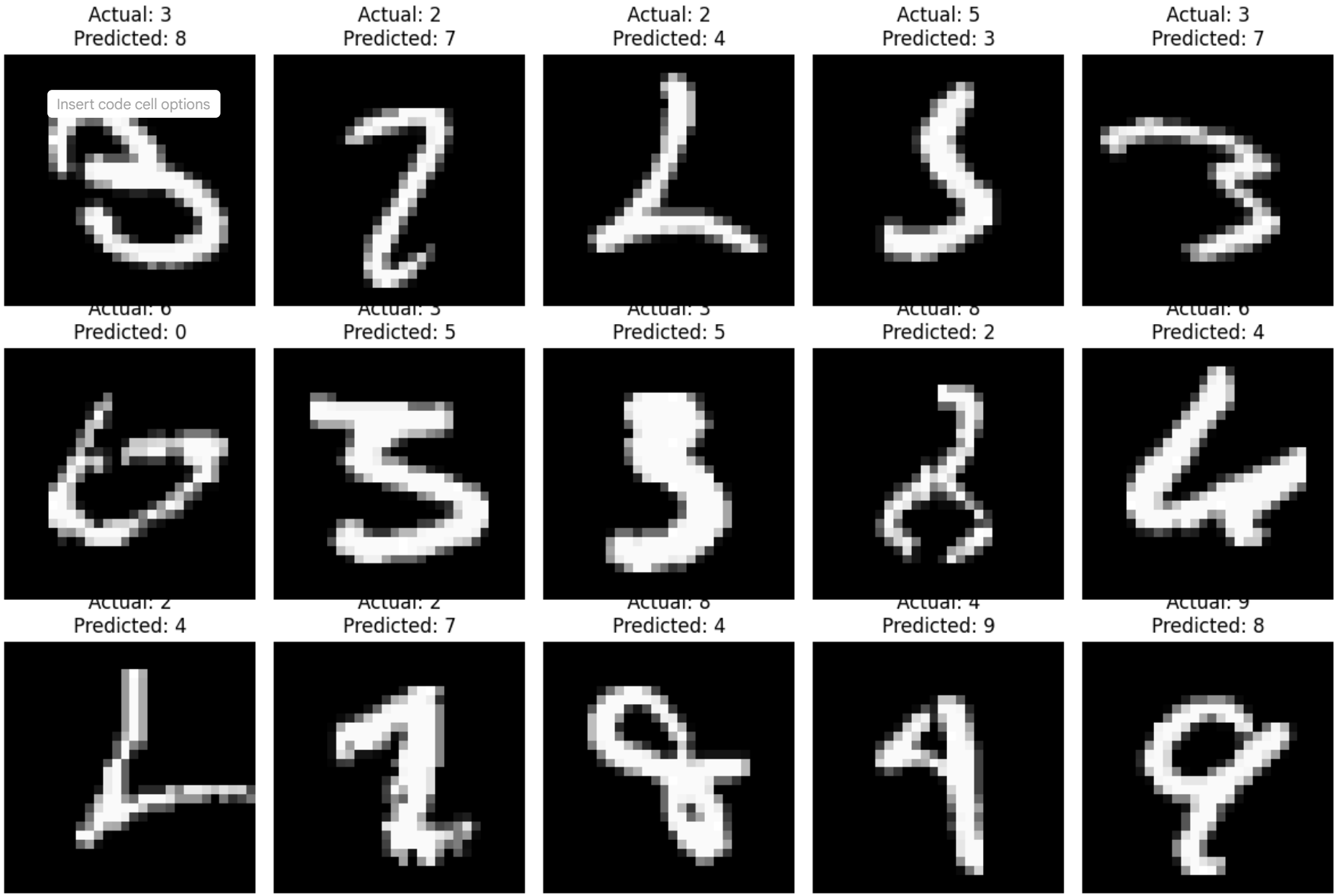
plt.figure(figsize=(12, 8))

for i in range(min(15, len(incorrect))):
    index = incorrect[i]

    plt.subplot(3, 5, i + 1)
    plt.imshow(x_test[index].squeeze(), cmap="gray")
    plt.title(
        f"Actual: {y_test[index]}\nPredicted: {y_pred[index]}"
    )
    plt.axis("off")

plt.tight_layout()
plt.show()
```

```
Number of incorrect predictions: 95
```



Start coding or generate with AI.

EXPERIMENT 10 Character, Digit, or Face Classification using Convolutional Neural Networks (CNNs)

THEORY QUESTIONS AND ANSWERS

Insert code cell options

Q1. What is image classification? How does it differ from object detection and image segmentation?

Answer: Image classification assigns a class label to an entire image. Object detection identifies objects and their locations using bounding boxes and class labels. Image segmentation assigns labels to individual pixels or regions of an image to identify the exact boundaries of objects.

Q2. Explain the architecture and working principle of a Convolutional Neural Network (CNN).

Answer: A CNN consists of convolutional layers, pooling layers, fully connected layers, and an output layer. Convolutional layers extract important features from images. Pooling layers reduce the spatial dimensions of feature maps. The extracted features are flattened and passed through fully connected layers. Finally, the output layer produces the predicted class probabilities.

Q3. What is the role of Convolutional Layers, Pooling Layers, and Fully Connected Layers in a CNN?